

**Embedded Linux
Final Mini Project
Group 17**



Adam Kolodziej, adkol23@student.sdu.dk

Embedded Linux Spring 2024
University of Southern Denmark (SDU)
Odense
June 2024

1 Discussion

1.1 General information

All the functional requirements are fulfilled.

As it was requested the wildlife camera run on Raspberry Pi OS. Last lectures required having Ubuntu installed but we rolled back to Raspberry Pi OS.

take_photo.sh shell script is utilized to take photos which are later saved to the disk. To simplify things, photos are saved to location which is used by Apache2 server. It gives an opportunity to have direct access to photos through the HTML website. There are no additional symbolic links etc. used for that. The RPi has access point configured.

ESP8266 runs MQTT publisher which transfers information when the sensor is triggered. Subscriber from the script running on Raspberry Pi uses *take_photo.sh* which takes a photo. Also additional data is saved in json file.

Every 5 min photo is taken using crontab which runs *take_photo.sh* periodically.

Motion detector works and checks if there is any movement registered in front of the camera. It is done using *motion_detect.py* python script. Two following photos are compared.

Rain detector runs on Raspberry Pico. Servo is cheap and not precise but it reacts every time button is pressed. As it was required the data is sent via USB serial to RPi. It's read then transmitted via MQTT to another script which interprets information and publishes back wiper angle - the first script subscribes to this topic and sends the data back to the Pico.

All relevant events are logged using rsyslog.

All the functionalities are configured to run after power failure - a lot of them are run as services when the RPi boots.

Time synchronization between the drone and wildlife camera was implemented using two scripts: *get_drone_ip.sh* and *update_time.sh*.

Secure copying process is set so no photos or json files are lost. It is done automatically when the drone connects to the wildlife camera's access point. JSON files are edited as requested.

WiFi link quality and signal level are logged when the drone is connected to camera's access point.

Photo annotation is performed using Ollama LLaVA version 7b. JSON files get additional info accordingly to annotation data.

JSON files are committed to a git.

1.2 Way of programming

The whole system was created to align as much as it could with the main tenets of the Unix Philosophy. It is possible to modify the system only changing the way scripts are utilized. Modularity gives a lot of freedom when creating the real-life option. Almost all the scripts are written to get some input variables, perform calculations, interpret data, do some actions based on the input and then output specific thing. This means that scripts are really universal and let us create similar systems which doesn't need to be identical.

1.3 Shell scripts

All the scripts written by us are in shell. We utilized *motion_detect.py* and used provided programs for Raspberry Pico and modified one for ESP3288. Shell scripts are simple and very efficient. They are also really good if we want to have easy to reuse parts of the system. To

have the scripts running from the beginning after bootup or after specific trigger, we used services.

Generally, writing the shell scripts was pretty convenient. It is quite easy to put them together and have working, easy to follow system. One of the disadvantage is that shell scripts are slow but you can get over that. However, programming shell scripts isn't the most pleasant. Even if there are tools like `jq` which is used to operate on the JSON data, sometimes it is hard to understand how the code should behave (especially without much experience with the shell scripting). It is also crazy that even the spacebar in the wrong place can stop the program from running. Many other languages have compilers which can point at the issue - with shell you don't have that help. We had several moments when we couldn't figure out what creates an error.

Another drawback is other languages like C, have API to work with the communication protocols. It's much easier to send data via serial ports etc. It's just more intuitive to use functions made to optimize the process rather than basic commands like in shell.

Next issue is a maintenance of the code. Scripts are handful to reuse but if you want to modify them or find the problem, they are extremely hard to work with. After a few months it can be challenging to come back to the project and quickly start working on it.

1.4 Scalability

The system can have many more wildlife cameras. It was written to be able to run multiple cameras separately and they won't influence each other as long as they don't have completely the same names etc. Every camera has its own WiFi network with its own ssh key which allows the connection only with specific devices. Drones, having required information, can fly by and connect to every single one of the cameras one by one. It is important that the drone is the one asking for the files transfer. It means that even if the connection is lost, the drone will receive some files. When coming back near the camera (the connection is regained) it will continue syncing the files automatically. So as I said, we can run a lot of cameras and just fly around with the drone without any issues. However, there is one really rare situation which can follow to overwriting the file. They are sync by names. It means that if the photos are taken at the exact same moment, one of them will be overwritten during synchronization.

1.5 Prioritization of actions

There is one thing which can be improved - 5 min photo with external trigger collision. To take a photo every 5 min, we used cronjobs. The external trigger is supposed to take a picture when something steps on the external pressure plate. Then, the picture is analyzed if there is also motion. If yes, the picture is saved to the database. The problem I want to describe occurs when the trigger is activated in the moment as "5 min" photo should be taken. Now, if there is no motion detected, the photo won't be saved. It should be fixed in the future. One of the way to do that is adding one more script which will process the data coming from motion detection part. If there is motion detected, the photo with *Motion* trigger will be saved. Otherwise typical "5 min" photo with *Time* trigger will be saved. This will make sure that we won't skip any "5 min" picture even if the motion will occur.

1.6 Security

The system has a few layers of security. As it was described in the main report, first it's necessary to connect to the access point. The connection is secured with the password and

the authentication method is set to WPA2-PSK. It protects network access and data transmission by using an AES (Advanced Encryption Standard). We also use firewall - nftables - we set it up to use only SSH or HTTP. To fight with brute force attacks we have 3-attempt rule implemented which will block the external IP address for 1 hour afterwards. The connection between drones and cameras is only possible through SSH. We had to have the connection done automatically (to be able to transfer data from cameras to drones automatically). To achieve that we created the pair of SSH keys (public and private) and copied the public one to the cameras. You can create the key without the password but it's not recommended and it wasn't done by us.

2 Conclusion

The project was really interesting because it deals with the real-life problem. It was satisfying to create something on actual hardware. Something that can actually be used. We had to figure out a lot of small steps and get to the point when everything can talk to each other and work as the system. As far as I understand we fulfilled all the major requirements. Obviously, there are some issues and fields to develop like I mentioned in the Discussion part. Nevertheless, it is possible to use it like that.